

CSE 3203 – Software Engineering

Lecture 06.5: Software Processes

Atanu Shuvam Roy



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd



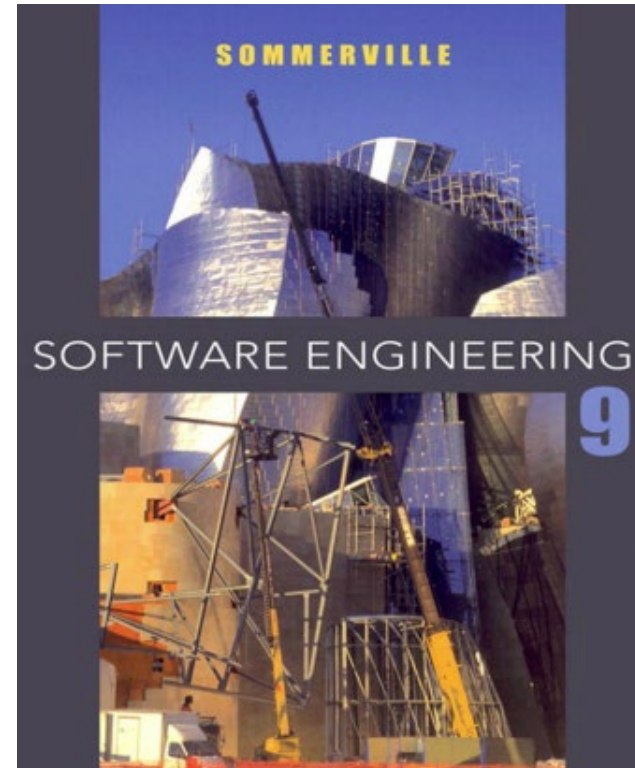
Class Code
byvb74r2



Fall 2025

Book for Reference

- Software Engineering 9th Edition by Ian Sommerville



Mark Distribution



- Mid Semester Exam – 25%
- Final Semester Exam – 40%
- Class Attendance & Performance – 10%
- Course Project – 25%

Is it possible to get A/A+ ?

- Absolutely. Just follow my instructions to the point.

How to fail in this course ?

- Don't follow my instructions.

Instructions



Topics covered

- ✧ Software process models
- ✧ Process activities
- ✧ Coping with change
- ✧ The Rational Unified Process
 - An example of a modern software process.

Process activities

- Real software processes are interleaved sequences of technical, collaborative, and managerial activities with the overall goal of specifying, designing, implementing, and testing a software system. Software developers use a variety of different software tools in their work. Tools are particularly useful for supporting the editing of different types of document and for managing the immense volume of detailed information that is generated in a large software project.
- The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved. How these activities are carried out depends on the type of software, people, and organizational structures involved. In extreme programming, for example, specifications are written on cards. Tests are executable and developed before the program itself. Evolution may involve substantial system restructuring or refactoring.

Requirements Eng Process

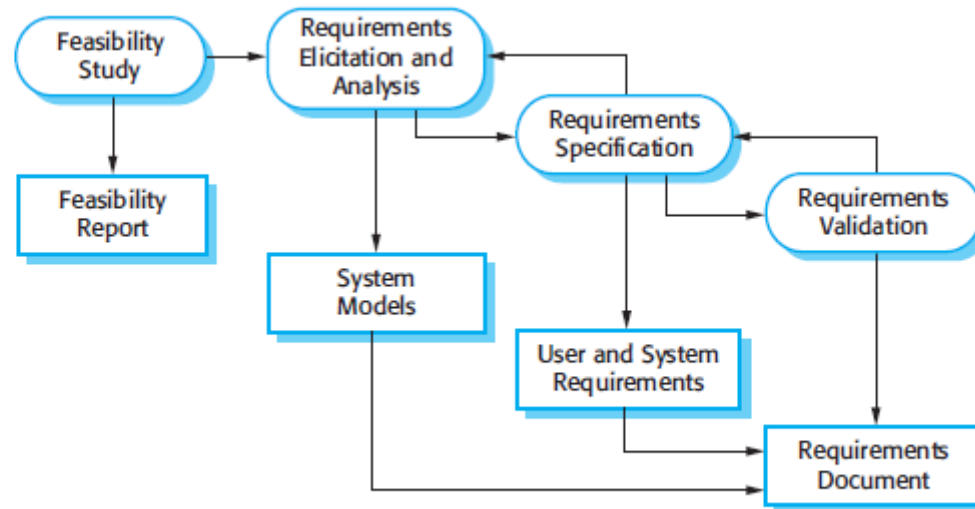


Figure 2.4 The requirements engineering process

Software Specification

- 1. *Feasibility study* An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints. A feasibility study should be relatively cheap and quick. The result should inform the decision of whether or not to go ahead with a more detailed analysis.
- 2. *Requirements elicitation and analysis* This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.

Software Design

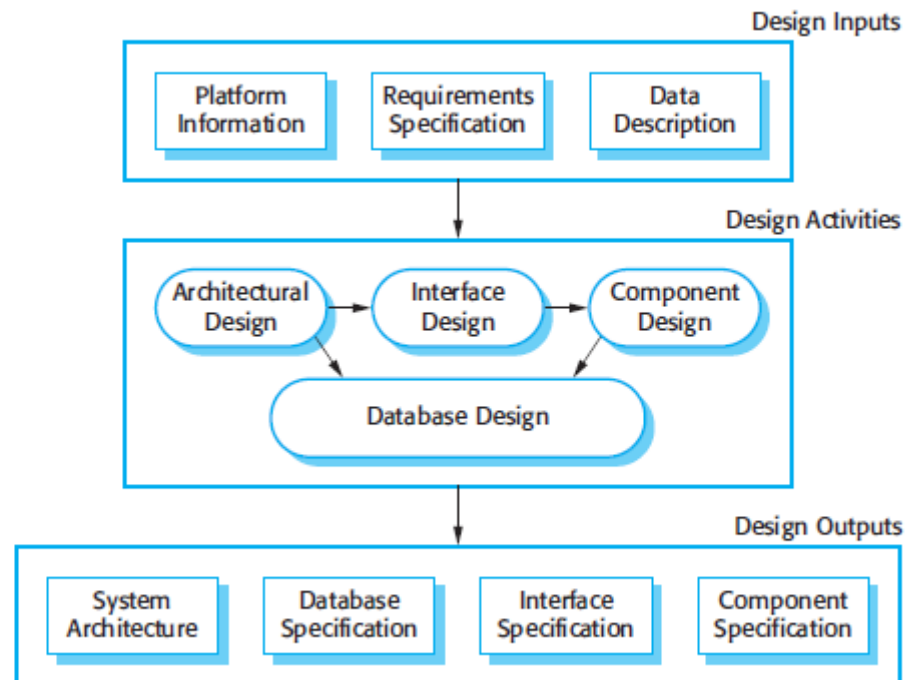


Figure 2.5 A general model of the design process

Design Process

- 1. *Architectural design*, where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships, and how they are distributed.
- 2. *Interface design*, where you define the interfaces between system components.

This interface specification must be unambiguous. With a precise interface, a

component can be used without other components having to know how it is

implemented. Once interface specifications are agreed, the components can be

designed and developed concurrently.

Design Process

- 3. *Component design*, where you take each system component and design how it will

operate. This may be a simple statement of the expected functionality to be

implemented, with the specific design left to the programmer.

Alternatively, it may

be a list of changes to be made to a reusable component or a detailed design model.

The design model may be used to automatically generate an implementation.

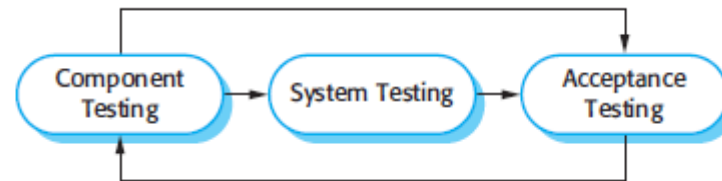
- 4. *Database design*, where you design the system data structures and how these are

to be represented in a database. Again, the work here depends on whether an

existing database is to be reused or a new database is to be created.

Validation

Figure 2.6 Stages of testing

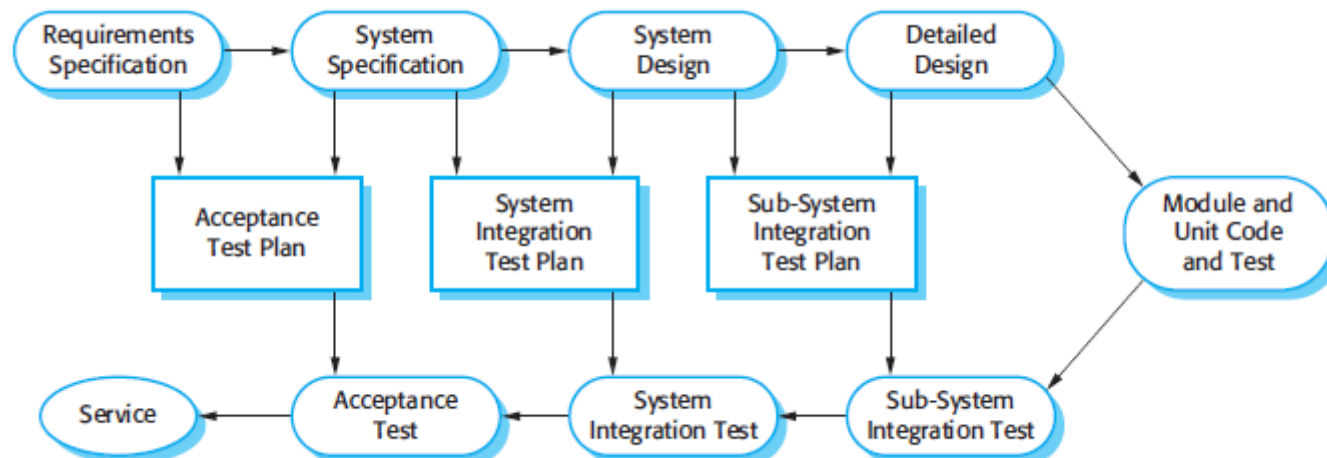


Validation

1. *Development testing* The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes, or may be coherent groupings of these entities. Test automation tools, such as JUnit (Massol and Husted, 2003), that can re-run component tests when new versions of the component are created, are commonly used.
2. *System testing* System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties. For large systems, this may be a multi-stage process where components are integrated to form subsystems that are individually tested before these sub-systems are themselves integrated to form the final system.

Validation

3. *Acceptance testing* This is the final stage in the testing process before the system is accepted for operational use. The system is tested with data supplied by the system customer rather than with simulated test data. Acceptance testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data. Acceptance testing may also reveal requirements problems where the system's facilities do not really meet the user's needs or the system performance is unacceptable.



Software Evolution

- The flexibility of software systems is one of the main reasons why more and more software is being incorporated in large, complex systems. Once a decision has been made to manufacture hardware, it is very expensive to make changes to the hardware design. However, changes can be made to software at any time during or after the system development. Even extensive changes are still much cheaper than corresponding changes to system hardware.

Software Evolution

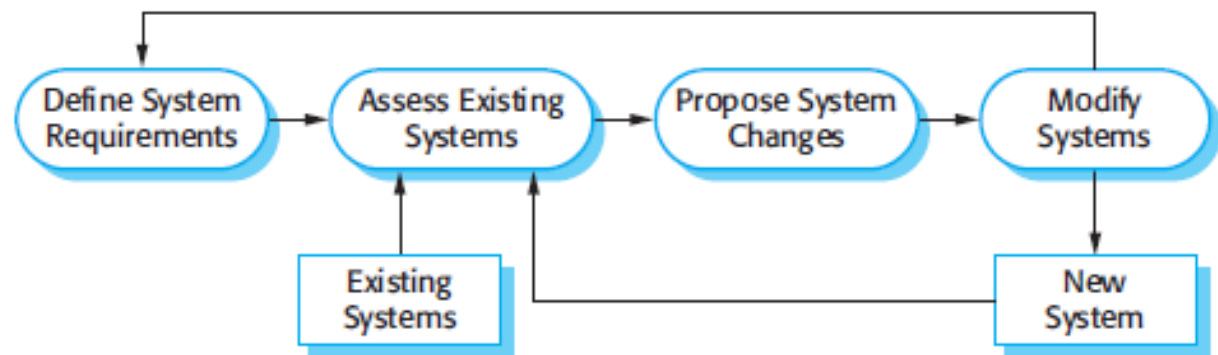


Figure 2.8 System evolution

Coping with Change

- There are two related approaches that may be used to reduce the costs of rework:
- 1. Change avoidance, where the software process includes activities that can anticipate possible changes before significant rework is required.
- For example, a prototype system may be developed to show some key features of the system to customers. They can experiment with the prototype and refine their requirements before committing to high software production costs.
- 2. Change tolerance, where the process is designed so that changes can be accommodated at relatively low cost. This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have to be altered to incorporate the change.

Coping with change

- 1. System prototyping, where a version of the system or part of the system is developed

quickly to check the customer's requirements and the feasibility of some design

decisions. This supports change avoidance as it allows users to experiment with the

system before delivery and so refine their requirements. The number of requirements

change proposals made after delivery is therefore likely to be reduced.

- 2. Incremental delivery, where system increments are delivered to the customer for

comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the whole system and allows changes to be incorporated into later increments at relatively

low cost.

Prototype Development

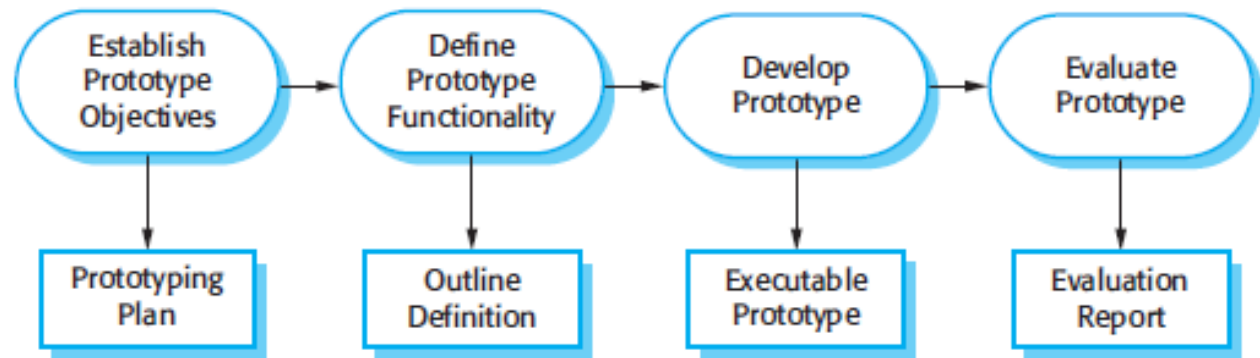
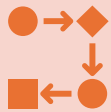
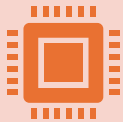


Figure 2.9 The process of prototype development

Prototyping



A prototype is an early version of a software system built to test ideas, explore designs, and understand problems before full-scale development. Rapid, iterative prototyping keeps costs low and allows stakeholders to experiment early in the process. It helps refine requirements during engineering, explore solutions in system design, and improve user interfaces through feedback.



Prototypes let users experience the system, suggest new requirements, and uncover errors or gaps in initial specifications. They are also used to test design feasibility—such as database performance or interface usability—since dynamic aspects of interfaces can't be captured well through text or diagrams.



The prototyping process involves setting clear objectives (e.g., testing feasibility, validating functionality, or designing a user interface), defining key functions while omitting less critical features to save time and cost, developing the prototype, and evaluating it with users. Evaluation includes training and observing normal use to identify further refinements.

Prototyping



However, prototypes can mislead if users or conditions differ from real-world use, or if managers push to reuse a throwaway prototype. Such reuse is risky because prototypes often lack documentation, ignore non-functional requirements, and have weak structure and quality.



Not all prototypes need to be executable—simple paper-based or “Wizard of Oz” prototypes can effectively test user interaction and design concepts at minimal cost.

Incremental Development

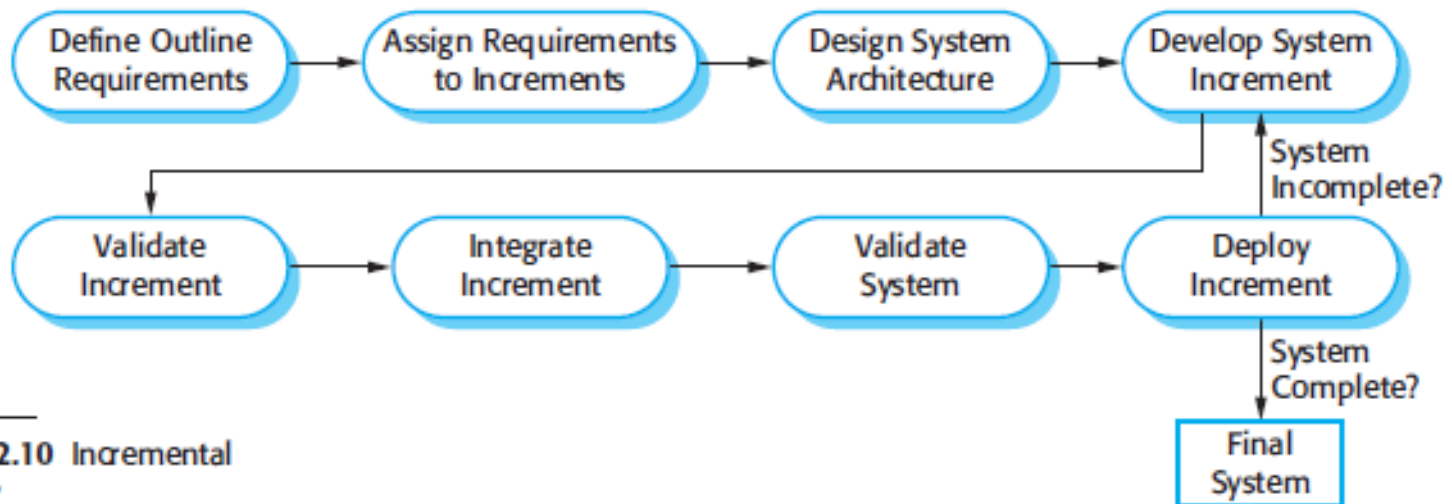


Figure 2.10 Incremental delivery

Incremental Development

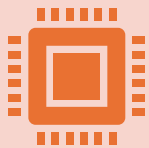


Incremental delivery is a development approach where a system is built and delivered in parts, or *increments*, each providing a portion of the overall functionality. Customers first outline the services they need and prioritize them. Development then proceeds in stages, delivering the most critical features first. Each increment is developed, tested, and deployed independently, allowing users to gain early access to working parts of the system while shaping requirements for future increments.



This approach lets customers use early versions as both prototypes and functional tools, helping refine requirements without wasted effort. It also provides immediate value, ensures that high-priority features are thoroughly tested, and makes incorporating future changes easier.

Incremental Development



However, incremental delivery has challenges. Shared system components can be hard to design when full requirements aren't known early. Users replacing an old system may resist using partial versions. Additionally, traditional procurement models often demand complete specifications upfront, which conflicts with the evolving nature of incremental delivery—especially for government or large-scale contracts.

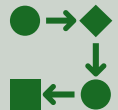


Incremental methods may not suit very large, distributed, embedded, or safety-critical systems, where full analysis of all requirements is essential. In such cases, an iterative prototyping process can still be used to explore requirements and designs before finalizing the complete system specification.

Boehm's spiral model



Boehm's Spiral Model is a risk-driven software process framework that represents development as a spiral rather than a linear sequence. Each loop of the spiral corresponds to a phase in the project—such as feasibility, requirements, or design—and includes activities to identify and manage risks before moving forward. The model blends *change avoidance* with *change tolerance* by treating change as a response to risk.

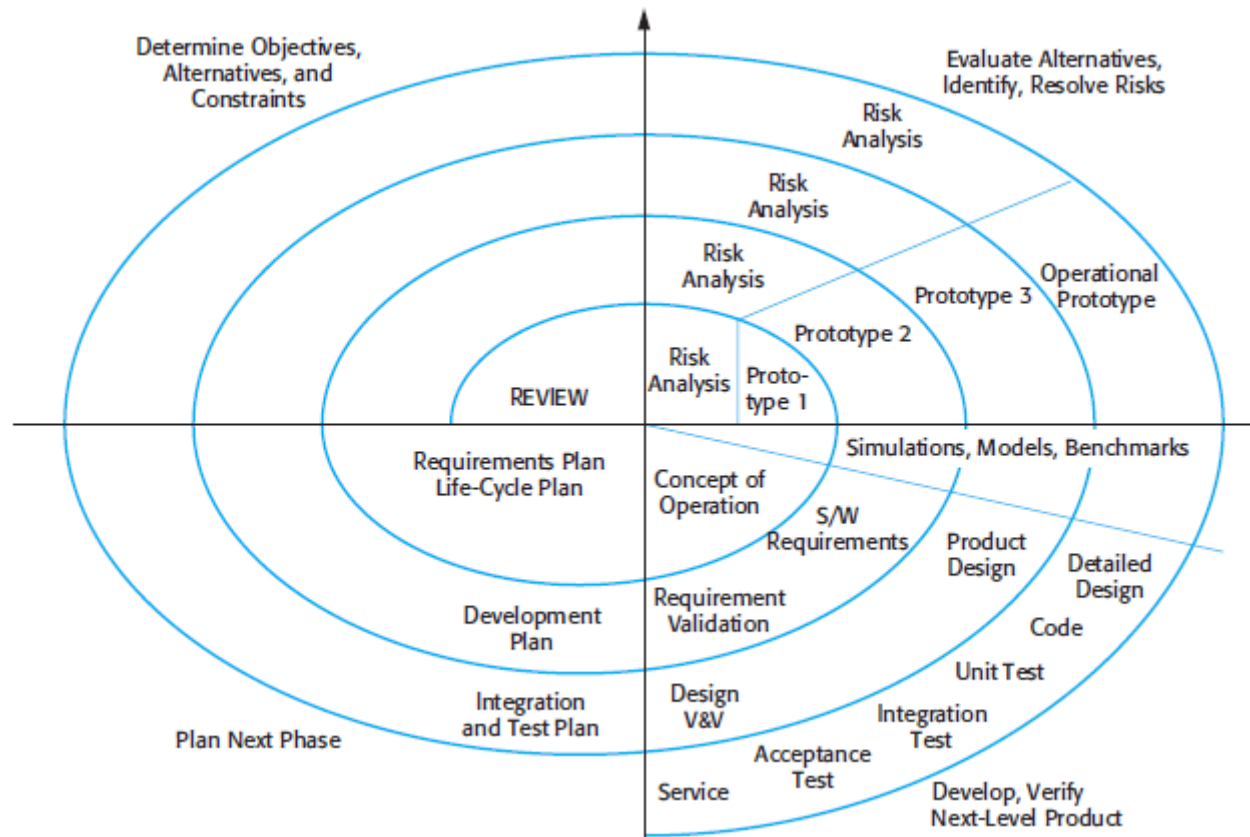


Each loop in the spiral includes four stages: **objective setting** to define goals and plans, **risk assessment** to identify and mitigate key risks, **development and validation** to choose and apply the best model based on those risks, and **planning** to review progress and prepare for the next phase.



Overall, the spiral model emphasizes iterative development with continuous risk evaluation, making it especially suited for complex, high-risk software projects.

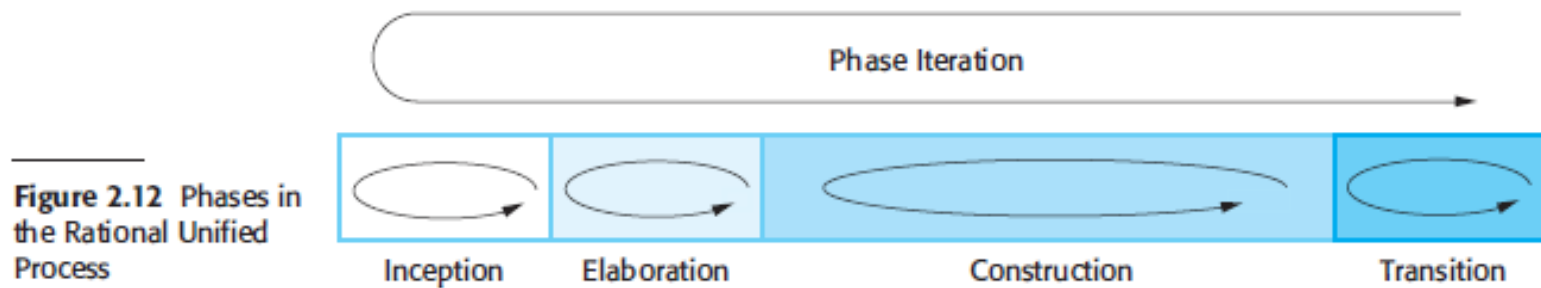
Boehm's spiral model



The Rational Unified Process

- The **Rational Unified Process (RUP)** is a modern, hybrid software process model derived from the Unified Modeling Language (UML) and the Unified Software Development Process. It combines principles from traditional models, supports prototyping and incremental delivery, and emphasizes best practices in design and specification.
- RUP presents the development process from three perspectives: a **dynamic view** showing its phases over time, a **static view** describing ongoing workflows, and a **practice view** outlining recommended development practices. It defines four key **phases**: **Inception** (establish the business case and system scope), **Elaboration** (define requirements, architecture, and plans), **Construction** (develop, test, and integrate the system), and **Transition** (deploy and stabilize the product in the user environment). Each phase can be iterative, and the entire process can loop back for continuous refinement.

Rational Unified Process (RUP)



KEY POINTS

- Software processes are the activities involved in producing a software system. Software process models are abstract representations of these processes.
- General process models describe the organization of software processes. Examples of these general models include the waterfall model, incremental development, and reuse-oriented development.
- Requirements engineering is the process of developing a software specification. Specifications are intended to communicate the system needs of the customer to the system developers.
- Design and implementation processes are concerned with transforming a requirements specification into an executable software system. Systematic design methods may be used as part of this transformation.
- Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- Software evolution takes place when you change existing software systems to meet new requirements. Changes are continuous and the software must evolve to remain useful.
- Processes should include activities to cope with change. This may involve a prototyping phase that helps avoid poor decisions on requirements and design. Processes may be structured for iterative development and delivery so that changes may be made without disrupting the system as a whole.
- The Rational Unified Process is a modern generic process model that is organized into phases (inception, elaboration, construction, and transition) but separates activities (requirements, analysis, and design, etc.) from these phases.



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering